SPE-229343-MS

Reinforcement Learning-Based Process Control in the Oil and Gas Industry

F. S. Alqaydi, ADNOC Onshore, Abu Dhabi, UAE

This paper was prepared for presentation at the ADIPEC held in Abu Dhabi, UAE, 3 – 6 November 2025.

Abstract

This paper explores the application of reinforcement learning for process control in an oil plant, aiming to enhance operational efficiency and stability. The project develops a custom simulation environment, replicating a crude oil separation processing plant. The reinforcement learning algorithms were implemented to manage the entire plant, aiming to maintain stable operation and maximize production. A comparison between traditional control methods and reinforcement learning based control was conducted and the results demonstrate that reinforcement learning can achieve comparable or superior performance to traditional controllers in managing process variables across multiple interconnected components. The paper contributes to the growing body of work in applying reinforcement learning to industrial process control, highlighting its potential to automate complex real-time operations.

Introduction

Process control in oil processing plants has traditionally been controlled by Proportional-Integral-Derivative (PID) controllers, as they are known for their simplicity and reliability. Despite their popularity, PID controllers have some limitations, especially when dealing with complex, nonlinear systems. As oil plants become more complex, there is a growing need for advanced control methods that can adapt dynamically to changing conditions, optimize performance, and reduce human intervention. Reinforcement Learning (RL), a branch of machine learning, offers a promising alternative to traditional control systems. RL algorithms learn through interaction with an environment, receiving feedback in the form of rewards or penalties, and iteratively improving their decision-making strategies. This research investigates the application of RL for controlling an oil plant, focusing on the development and implementation of custom RL environments and algorithms. The study explores the effectiveness of two RL algorithms Proximal Policy Optimization (PPO) and Deep Deterministic Policy Gradient (DDPG) in managing the operations of a simulated oil processing plant. By comparing the performance of RL based control systems with traditional PID controllers, this research aims to demonstrate the potential of RL to enhance process control in industrial settings. The paper begins by outlining the oil separation plant process and the use of conventional PID controllers, before introducing reinforcement learning and related research. It then details the development and implementation of RL agents within a plant simulation, alongside the design of a graphical user interface. The study compares the performance of RL algorithms against PID controllers, concluding with a discussion of results, potential improvements, and directions for future work

Technical Background

Crude Oil Separation Plant Overview

Crude oil is typically a mixture of three mediums: oil, natural gas, and water. The plant model in this project is based on a simple crude separation process. At the beginning of the process, oil wells, each with different oil/gas/water ratios, feed into the processing plant. The fluid first settles in a separator, where separation occurs based on specific gravity differences oil, being less dense at 0.85 g/cm^3, rises, while water at 1 g/cm^3 settles at the bottom of the vessel and gas accumulate at the top. Next, the fluid moves to the desalter, where fresh water is injected, and an electric field is applied to remove salt and enhance purity. Following this, the liquid enters a stabilization column, where it is heated to further separate the three mediums. Finally, the separated oil is stored in a tank for export. The gas from the separator and stabilization column is directed to the gas processing train, which consists of compressors in series that increase the gas pressure, allowing it to be re-injected into reservoirs to enhance oil recovery. Water is treated and safely disposed of. The entire process is controlled by monitoring process variables within the system, ensuring efficient separation and processing. Figure 1 shows an overview of the process.

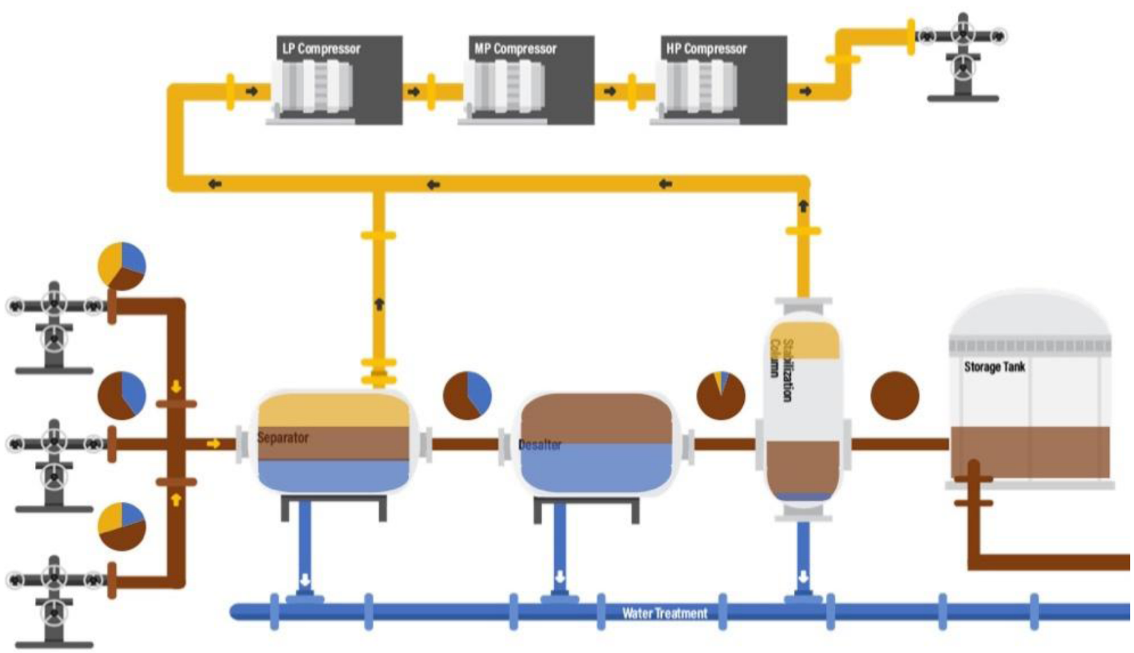

Figure 1—Overview of the crude oil separation process and main plant components.

Proportional-Integral-Derivative Controllers

Oil plants have traditionally been controlled by PID controllers, which operate as a closed feedback loop. The controllers calculate the error between a set-point and the actual value, then apply a correction based on three terms:

- Proportional (P): Adjusts the input in proportion to the error.
- Integral (I): Accounts for accumulated past errors to reduce steady-state error.
- Derivative (D): Predicts future errors by considering the rate of change, providing a damping effect.

This correction is then applied to a control element such as a valve to minimize the error effectively (Åström and Hägglund, 2006). PID controllers are widely used in industry due to their simplicity and robustness, rooted in mathematical formulas that ensure predictable outputs. Their computational efficiency enables fast response times, which is crucial in oil processing plants for quickly correcting deviations and maintaining stability in dynamic environments. Additionally, the relatively low cost of PID controllers makes them the prime choice for industrial applications. However, like all technologies, PID controllers have limitations. Traditional PID controllers often have a limited control scope, typically managing only one process variable. They also struggle with complex systems. Tuning these controllers and selecting the correct parameters can be challenging, and incorrect tuning can lead to process instability and sluggish response. Lastly, PID controllers can only maintain a process at a set-point after manual startup and stabilization is done as they offer no startup or shutdown capabilities.

Reinforcement Learning

Reinforcement Learning is a distinct type of machine learning. In RL, no data is fed to the model, neither labeled nor unlabeled. Instead, the model learns from interacting with an environment over multiple episodes with the goal of maximizing a reward function (Sutton and Barto, 2018). This approach is inspired by the nature of how a human learns through experience. This method is particularly beneficial when no previous data is available, and no clear solution is known in advance. As the agent learns from experience, it has an advantage in tasks that require sequential decision-making (Arulkumaran et al., 2017).

Literature Review

The advancements in process control using RL have seen increasing attention over recent years. RL is uniquely positioned to tackle complex control problems where traditional methods fall short. This literature review examines recent progress in applying RL to process control, exploring both theoretical advancements and practical implementations.

Process control using reinforcement learning is not a new concept Hopkins and Himmelblau pioneered this field back in 1992. They proposed a novel approach where RL is used instead of traditional methods Their study highlighted the potential of RL to control processes in the absence of a clear objective function, similar to how a human learns through trial and error. They demonstrated RL applicability to non-linear systems like Continuous Stirred Tank Reactors. Understandably the training was time consuming, but the results were comparable to traditional PID controllers (Hopkins and Himmelblau, 1992).

(Vamplew et al., 2021) explored the usage of RL in process control, focusing on the flexibility of RL in dynamic environments. By comparing RL's performance with traditional control methods, they showed the advantages of RL in handling non-linear and complex processes and its potential to exceed traditional methods. They also presented the challenges with applying RL in real-world industrial environments, such as the difficulty of ensuring safety and reliability.

(Gladwin et al., 2021) provided a broad review of RL applications in process industries, showing the potential of RL in industrial process control. The review covered many RL algorithms, their strengths, and their use in different types of process controls. The authors discussed the challenges in using RL, such as the need for comprehensive training and computational power. Nevertheless, they concluded that the benefits, including improved efficiency, flexibility, and the ability to manage complex, multi-variable processes, outweigh the challenges.

(Narendra and Dutta, 2021) introduced the use of RL as a control system, this paper was aimed at beginners in the field. They outlined the basic concepts of RL and showed how it could be applied to simple process control tasks. The paper served as a guide, offering explanation on model training, simulation, and real-time deployment. This paper makes it easy for any engineer or researcher new to the field to enter it.

(Williams et al., 2021) offered an overview of the latest progress in RL for process control. They discussed how advancements in RL, including improvements in algorithm efficiency and stability, made it more viable for real-time process control applications. The paper reviewed several case studies where RL was successfully implemented in industrial settings. The authors concluded that while challenges remain, such as ensuring safety and interpretability, RL's potential to revolutionize process control is undeniable.

A huge leap was achieved by Yokogawa in 2022 when they have successfully controlled a desalination column for 35 days fully using RL. During this period, the RL system managed to maintain product quality and process stability while reducing energy usage, marking a significant milestone in the journey towards fully autonomous industrial operations (Yokogawa Electric Corporation, 2022a). This was done by Factorial Kernel Dynamic Policy Programming (FKDPP) algorithm developed in collaboration with Japan's Nara Institute of Science and Technology (NAIST). FKDPP offers a robust RL control model that can be generated within just 30 learning trials (Yokogawa Electric Corporation, 2022b).

(Lee et al., 2021) presented another approach in using RL in process where RL is used for process design, RL is utilized to optimize design parameters. They highlighted case studies where RL was used to optimize complex systems with multiple objectives, showing RL's ability to find solutions that balance competing goals like cost, efficiency, and safety. The review also identified areas where further research is needed, particularly in developing RL algorithms that can handle the high dimensionality and complexity of process design tasks.

The literature shows a growing interest in adopting RL for process control, driven by its ability to manage complex, non-linear, and dynamic systems more effectively than traditional methods. Additionally, RL holds the potential for achieving full autonomy in industrial operations and increasing process safety and profitability. Although challenges such as simulation requirements and computational costs remain, continuous advancements in this field are expected outweigh the negatives, leading to the widespread adoption of RL in process control.

Research Contribution

The overwhelming majority of the work done in this area is focused on using RL to control a single equipment. This is done to increase the performance of a certain part of the process, but this approach result is a similar behavior to traditional control method because of the limited control scope of a single equipment. In this research the RL aims to control a whole processing plant which will result in an interesting control dynamic where the whole system is proactively working to reach the stability and maximum production. The RL results are benchmarked against PID control scenarios to measure the effectiveness of RL against current industry standards.

Methodology and Implementation

Simulation Development

The simulation was developed using the SimPy library, a process-based discrete-event simulation framework in Python. SimPy allows the definition of concurrent components using generator functions, which are executed within an event-driven simulation environment. This made it an ideal choice for modelling the dynamic behavior of oil and gas process systems.

To organize the simulation logic, custom classes that represent key components of the process plant were created. These include:

- **Well:** Simulates a production well delivering a multiphase flow (oil, water, gas) regulated by a choke valve.

- **Vessel:** Represents a generic process vessel (separator, desalter, stabilization column, or storage tank) that manages inflows and outflows of different fluid phases while tracking internal levels and pressure conditions. Every vessel has oil, water and gas outlet valves
- **Compressor:** Models a gas compressor that increases pressure before transferring gas downstream or to a flare system when not operational.
- **Flare:** Simulates a safety unit responsible for safely burning excess gas, preventing pressure build-up in the system.

Each class operates as an independent SimPy process, continuously interacting with others through shared state and fluid transfers. The modular structure of the simulation facilitates flexibility and scalability, enabling the integration of additional process equipment or control logic as required.

Environment Development

Building on the simulation infrastructure, a custom reinforcement learning environment was developed using the OpenAI Gym interface. This environment leverages the previously defined simulation classes and enables seamless integration with RL algorithms.

The environment encapsulates the behavior of the entire plant, including wells, vessels, compressors, and flare systems. Its key components include:

- **Action space:** The action space is continuous, ranging from 0 (valve fully closed) to 1 (fully open), covering all controllable elements such as choke valves and vessel outflow valves.
- **Observation space:** The observation space captures the current water and oil levels, along with gas pressures across the vessels.
- **System Configuration:** The environment initializes a SimPy simulation with three wells, four interconnected vessels, three compressors, and a flare. Components are linked to reflect the logical flow of oil, water, and gas through the process plant.
- **Reset and Step Functions:** The reset method reinitializes the environment state, while the step function executes agent actions, advances the simulation by one time step, and returns updated observations. Episodes terminate after a fixed number of steps.
- **State Representation and Reward Mechanism:** The environment's state is represented as a vector of water levels, oil levels, and gas pressures. The reward function penalizes deviations from predefined water and oil setpoints using a quadratic loss. This was done to highly penalize large deviation from the setpoint while being more forgiving for small deviation, gas pressure is penalized only when exceeding a threshold. This design encourages stable operation around optimal levels and ensures all process variables are always within safe operating envelope.

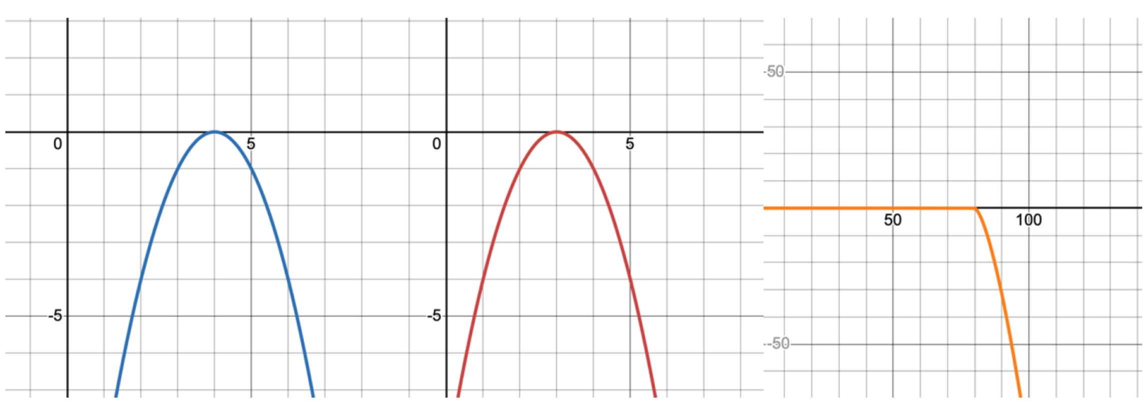

Figure 2—Reward function design illustrating penalties for deviations in oil, water, and gas process variables.

This setup results in a fully integrated, dynamic environment tailored for training RL agents to perform real-time control in an oil and gas production setting.

Reinforcement Learning Implementation

With the environment structure established, the next step involved applying reinforcement learning to control the simulation. Selecting appropriate RL algorithms is a critical consideration, especially given the complexity of the environment. RL methods are generally divided into model-based and model-free approaches. Due to the dynamic, high-dimensional nature of the plant model, model-free algorithms were deemed more suitable.

Two prominent model-free algorithms were selected:

- **Proximal Policy Optimization (PPO):** A stochastic, policy-based method known for its stability and effectiveness in continuous action spaces.
- **Deep Deterministic Policy Gradient (DDPG):** A deterministic actor-critic method that combines policy-based and value-based learning, offering fine-grained control over continuous actions.

These algorithms were chosen not only for their performance in continuous control tasks, but also to compare stochastic (PPO) versus deterministic (DDPG) learning behaviors.

Proximal Policy Optimization. PPO employs two neural networks:

- The Policy Network, which outputs a probability distribution over possible actions based on the current state. This enables the agent to explore the environment by sampling different actions with varying likelihoods.
- The Value Network, which estimates the expected return (cumulative reward) from a given state. This estimate is essential for computing the advantage function, which guides the policy updates.

Training proceeds by collecting interaction data, computing advantages, and updating both networks. PPO's clipped objective function ensures stable learning by avoiding large, destabilizing updates to the policy.

PPO Training Workflow:

1. Initialize policy and value networks.
2. Collect state–action–reward trajectories through interaction with the environment.
3. Compute returns and advantage estimates.
4. Update the policy network using a clipped surrogate objective.
5. Update the value network by minimizing prediction error.
6. Repeat the process over multiple iterations until convergence.

Deep Deterministic Policy Gradient. DDPG is an actor-critic algorithm designed for continuous action spaces. It comprises:

- An Actor Network that maps states directly to deterministic actions.
- A Critic Network that evaluates the Q-value of each state–action pair, guiding the actor towards more rewarding policies.

DDPG also maintains target networks (delayed copies of the actor and critic) to stabilize training. Additionally, experiences are stored in a replay buffer, allowing the agent to learn from randomly sampled past interactions, improving sample efficiency.

DDPG Training Workflow:

1. Initialize actor and critic networks, along with target networks.

2. Interact with the environment to gather experiences.
3. Store experiences in a replay buffer.
4. Sample mini-batches for training.
5. Update the critic network using the Bellman equation.
6. Update the actor network using gradients from the critic.
7. Soft-update the target networks.
8. Repeat across episodes until desired performance is achieved.

Together, these two algorithms provide complementary approaches for training intelligent agents to control complex multiphase processes. Their implementation enables comparative evaluation of stochastic versus deterministic control in continuous industrial environments.

Hyperparameter Tuning

Hyperparameters are critical settings that define how reinforcement learning models are trained. Unlike model parameters, they are set before training and significantly impact performance. This project involved tuning key hyperparameters for both PPO and DDPG to improve learning efficiency and stability.

Optuna was used to automate hyperparameters tuning, Optuna is an efficient optimization library that tests and refines hyperparameter combinations. It defines an objective function, suggests values, evaluates performance, and prunes poor trials to save resources. The best configurations were selected after multiple trials.

Training

The training process was implemented using the Stable Baselines3 (SB3) library, a widely adopted framework for reinforcement learning in Python. SB3 provides robust implementations of state-of-the-art algorithms through a streamlined and flexible API, making it ideal for applied RL tasks.

The following steps summarize the training workflow:

1. **Environment Initialization:** The custom environment simulating the plant process is created. It provides observations, receives agent actions, and returns rewards based on how effectively the agent maintains process setpoints.
2. **Hyperparameter Configuration:** The selected RL algorithm (PPO or DDPG) is configured with optimized hyperparameters based on the tuning results from Optuna.
3. **Model Training:** The RL agent interacts with the environment over multiple episodes:
    - Observations are used to predict actions.
    - Actions are executed, and the environment returns the next state and reward.
    - Rewards accumulate until the episode ends.
    - The model is updated iteratively using collected experience.
    - A moving average of the episode rewards (last 100 episodes) is tracked to monitor performance.
4. **Model Saving & Training Time:** After training, the model is saved, and the total training time is recorded to evaluate efficiency.
5. **Training Visualization:** Episode rewards and moving averages are plotted to visualize progress and ensure learning is stable.

Graphical User Interface (GUI) Development

To complement the simulation and reinforcement learning environment, a custom graphical user interface was developed to act as an operational test bench. The GUI provides real-time visualisation of process variables, including oil and water levels, gas pressures, and valve positions across all vessels and wells. It integrates three control modes: manual operation, PID-based control, and reinforcement learning, enabling direct benchmarking of the RL agents against conventional methods within the same interface.

The interface was designed to reflect industrial control system layouts, incorporating features such as live trend plots and valve state indicators. This ensures the tool not only aids in evaluating algorithm performance but also demonstrates the system's potential for real-world deployment. By offering an intuitive, plant-like control dashboard, the GUI bridges the gap between simulation research and practical process operations, highlighting the applicability of RL-driven control in oil and gas facilities.

Results and Discussion

In this chapter, the trained reinforcement learning models are evaluated based on their training performance and overall ability to control the simulation, with comparisons made to traditional PID controllers.

Model Training Evaluation

Reward Accumulation. Reward accumulation is a key indicator of the learning progress in reinforcement learning. Ideally, the cumulative reward should increase over time and eventually stabilize, signaling convergence to an optimal control policy.

Figures 3 and 4 illustrate the change in reward over 500 training episodes for the PPO and DDPG models, respectively. The PPO agent demonstrates quicker convergence and a smoother learning curve, whereas DDPG exhibits more variability during training. Despite these fluctuations, both models ultimately converge to a satisfactory level of performance, indicating that both algorithms successfully learned effective control strategies for the environment.

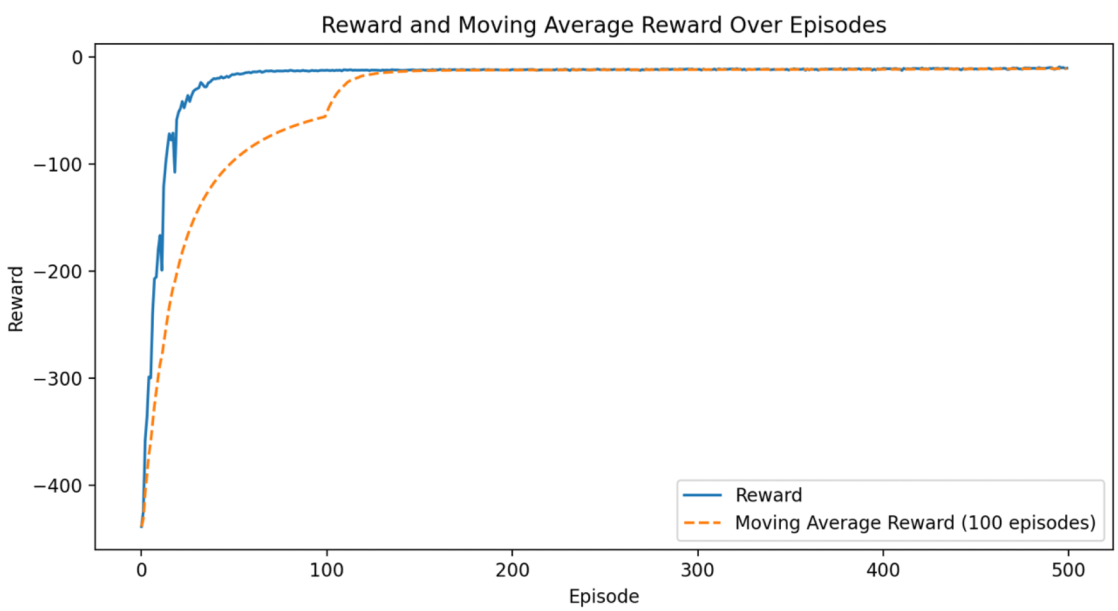

Figure 3—PPO training curve showing cumulative reward progression over 500 episodes.

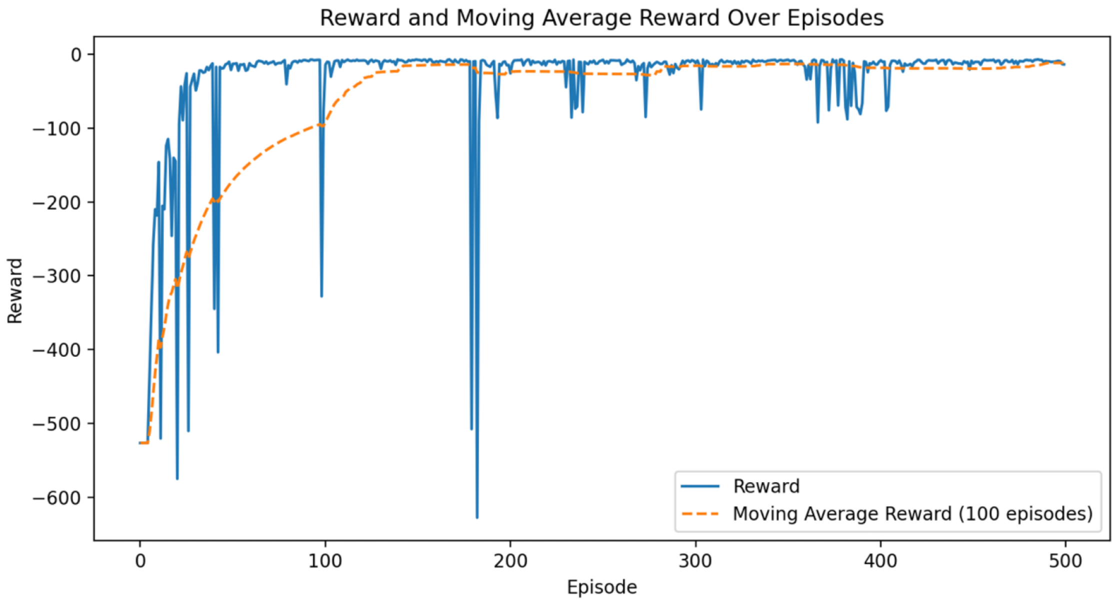

Figure 4—DDPG training curve showing cumulative reward progression over 500 episodes.

Exploration vs. Exploitation. The trade-off between exploration and exploitation is a fundamental aspect of reinforcement learning. In the early stages of training, agents typically engage in exploration to learn about the environment and discover potentially effective strategies. As training progresses, the agent shifts toward exploitation leveraging acquired knowledge to maximize rewards.

Entropy was used as a metric to measure this balance. Entropy quantifies the uncertainty in the agent's action selection: higher entropy values reflect more exploration, while lower values suggest that the agent is committing to specific actions based on learned behavior.

In Figure 5, the entropy of the PPO model is plotted over time. The decreasing trend in entropy clearly shows that PPO transitions from exploration to exploitation as training progresses. This behavior is expected due to PPO's stochastic nature and indicates healthy policy refinement over time.

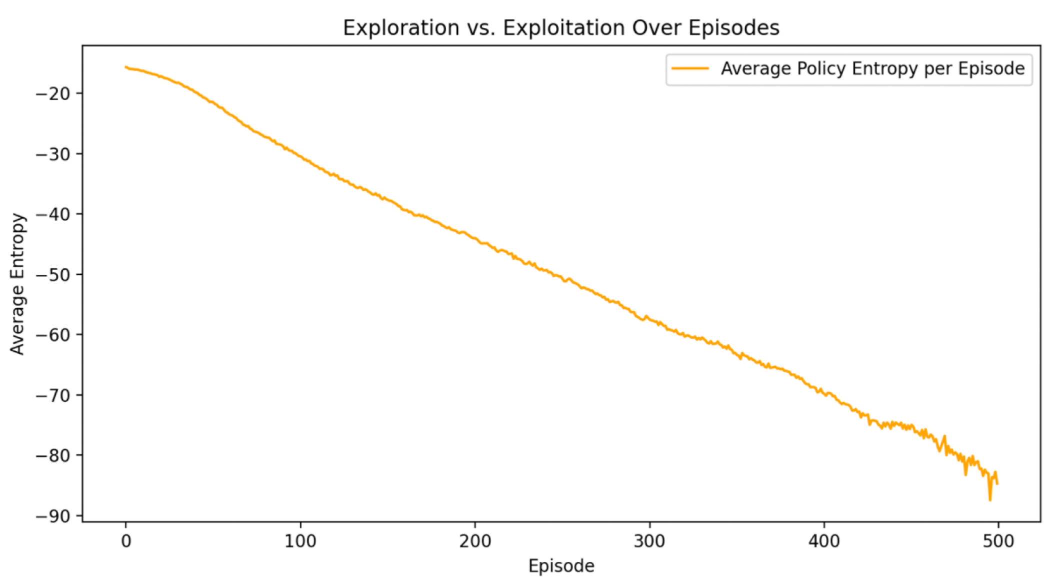

Figure 5—PPO entropy trend illustrating the shift from exploration to exploitation during training.

In contrast, Figure 6 shows the entropy trend for the DDPG model. Because DDPG is a deterministic algorithm, traditional entropy is not directly applicable. Instead, the standard deviation of the selected actions was tracked as a proxy for exploration. As no additional exploration noise was introduced during training, DDPG maintained relatively stable action patterns throughout. This stability reflects its tendency to exploit known strategies rather than explore new ones.

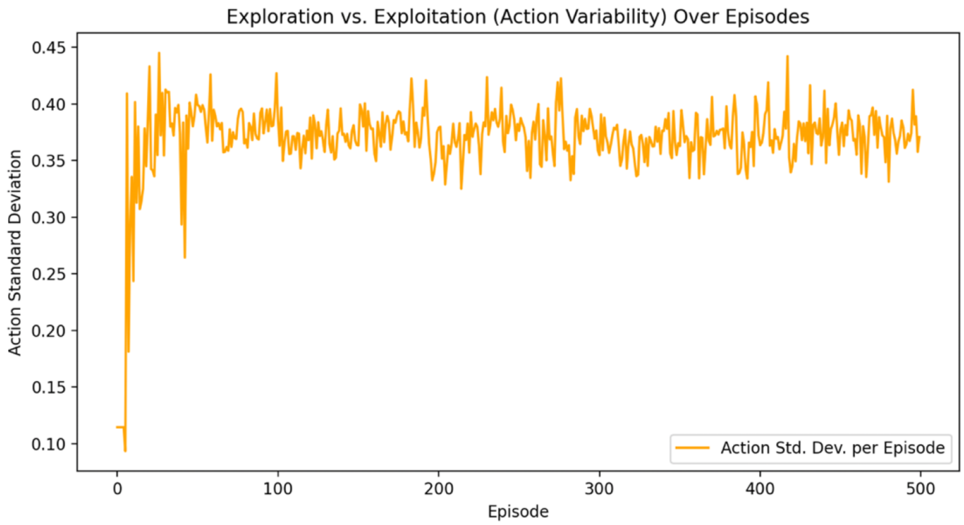

Figure 6—DDPG exploration behaviour represented by action variability over training episodes.

This behavior is acceptable and even advantageous in the context of this project. While reduced exploration could be a limitation during the plant's startup phase, it is beneficial once the plant reaches steady-state operation. The deterministic and stable control produced by the DDPG agent aligns well with the requirements of continuous process control, where consistency and predictability are critical.

Performance Against Traditional Control Methods

This section compares the performance of the trained RL agents against conventional PID controller. Evaluation is based on three key criteria: overall control behavior, cumulative deviation from setpoints, and system stability. The setpoints for each vessel are listed in Table 1.

Table 1—Target setpoints for oil, water, and gas variables across process vessels.

Vessel	Oil Level	Water Level	Gas Pressure
Separator	4	3	<80
Desalter	4	4	-
Stabilization column	5	0	0
Storage tank	5	0	-

Overall Control. To evaluate control quality, time-series graphs were generated for each vessel, displaying oil level (brown), water level (blue), and gas pressure (yellow) across time. The figures correspond to each vessel as follows: Separator (top left), Desalter (top right), Stabilization Column (bottom left), Storage Tank (bottom right)

As shown in Figure 7, the PID controller managed to regulate the process but required manual intervention during startup. Specifically, the oil inflow from the wells had to be reduced to prevent overload, highlighting PID's limited adaptability.

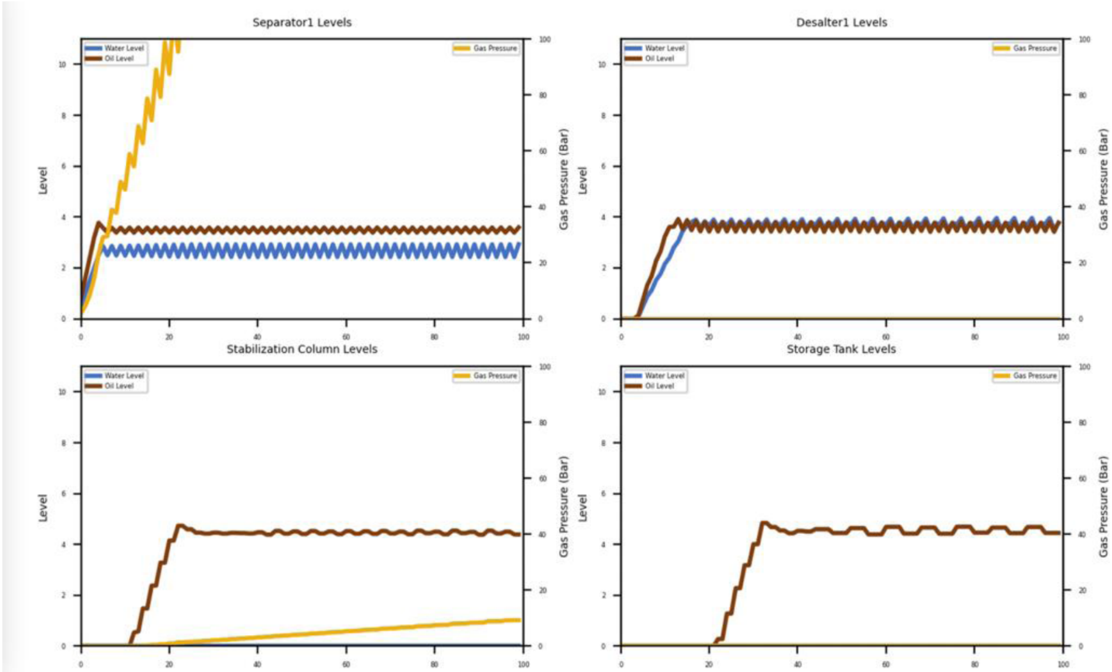

Figure 7—PID controller performance in regulating vessel levels and pressures.

The PID system produced oscillations around the setpoints, typical of feedback-only controllers in nonlinear systems. While it eventually stabilized, the path to steady-state was not smooth.

Figure 8 shows the PPO model's output. The agent initially struggled, with noticeable deviations during the first 30 time steps. However, once stabilized, PPO maintained steady levels across all vessels.

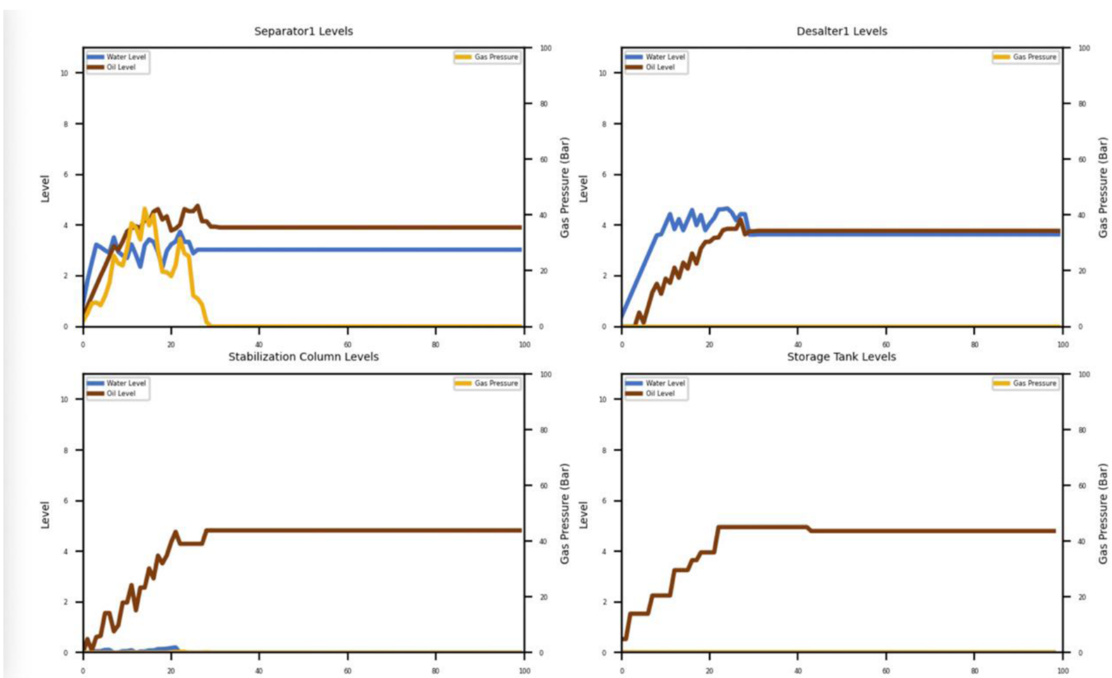

Figure 8—PPO controller performance

An unexpected fluctuation in the Storage Tank oil level at time step 40 demonstrates the stochastic nature of PPO and a potential drawback occasional unpredictable behavior which must be considered in safety-critical applications.

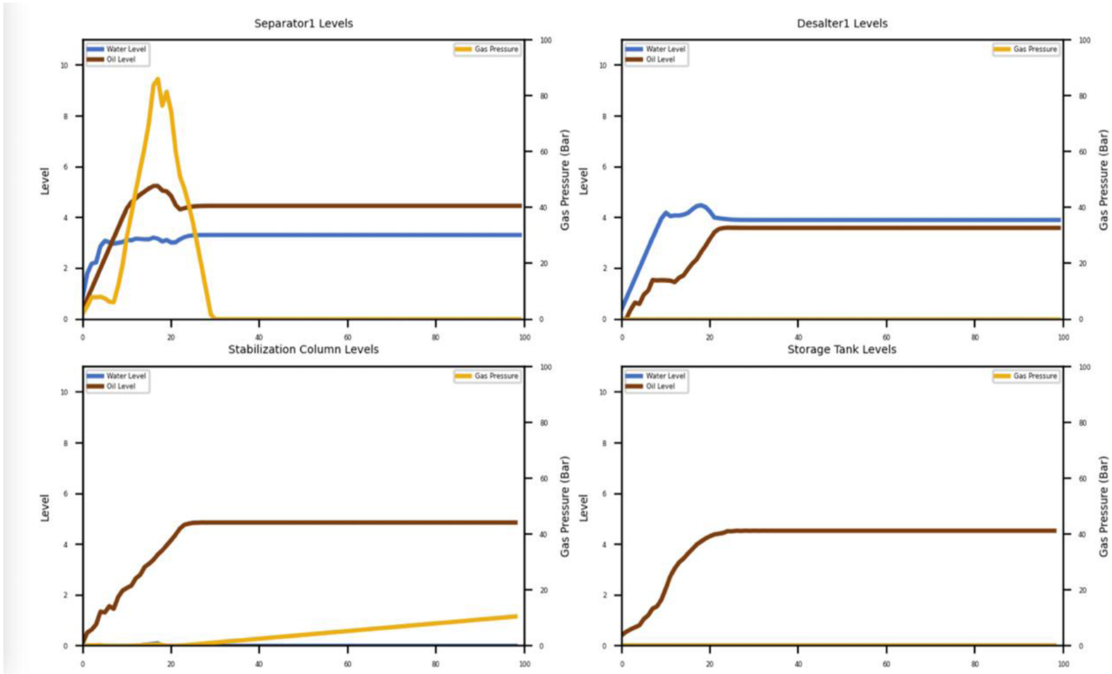

Figure 9—DDPG controller performance

The DDPG model outperformed both alternatives in terms of smoothness and stability. In the Separator, the model deliberately allowed an overshoot in oil level, anticipating the need to distribute fluid downstream.

This behavior reflects DDPG's strength in planning across time steps. Although a slight offset from setpoints was observed, this is acceptable given the quadratic reward function used, which tolerates minor deviations without heavy penalty.

In conclusion:

- PID required manual startup and resulted in a slightly oscillatory control.
- PPO achieved good performance after startup but exhibited one unpredictable action.
- DDPG provided the smoothest and most stable control, effectively coordinating fluid movement across all vessels.

Cumulative Deviation. To further evaluate controller performance, the cumulative deviation was calculated for each method. This metric was obtained by summing the absolute errors across all vessels over time, providing a comprehensive measure of control effectiveness.

Figure 10 shows the cumulative deviation curves for all three methods. As observed, both PPO and DDPG outperform the PID controller during the startup phase, reducing deviation more rapidly. While all controllers eventually achieve acceptable error levels, the RL-based methods maintain a slightly lower overall cumulative error.

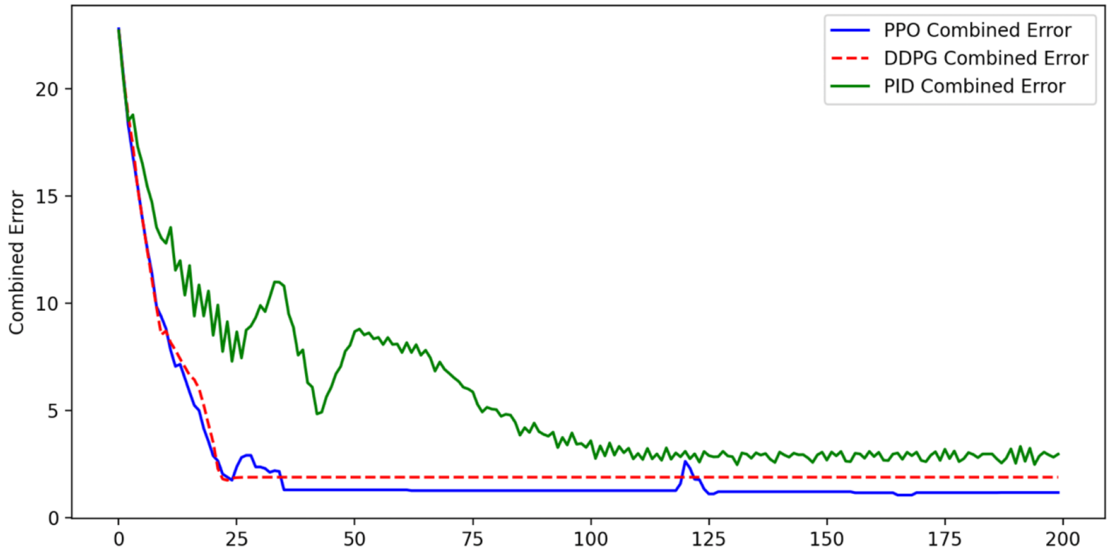

Figure 10—Cumulative deviation of process variables under PID, PPO, and DDPG controllers.

In addition to visual analysis, the area under the cumulative deviation curve was calculated as a scalar metric for evaluating long-term performance. These values are summarized in Table 2, including error reduction percentages relative to the PID baseline.

Table 2—Area under the cumulative deviation curve and percentage error reduction of PPO and DDPG relative to PID.

Control Method	200 Steps	Error Reduction	1000 Steps	Error Reduction
PID	1103	-	3776	-
PPO	396	64%	1636	56%
DDPG	545	51%	2230	41%

Error reduction is computed by comparing the difference between the PID cumulative error and the RL-based error, then dividing by the PID value. These results clearly demonstrate the superiority of RL models, with PPO achieving a 64% reduction in deviation over the first 200 steps.

Stability. Another important metric to measure is the stability of the system. Stability is quantified using the standard deviation, which measures how much the combined error fluctuates around its mean value. A lower standard deviation indicates more stable control, meaning the system's performance is more consistent with fewer oscillations around the setpoints.

As can be seen in table 3 both RL control methods outperformed the PID controller by having lower standard deviation which means higher stability.

Table 3—Standard deviation of errors for each vessel, comparing PID, PPO, and DDPG stability.

Control Method	Separator	Desalter	Stab Column	Storage Tank	Total Standard Deviation
PID	0.2817	1.2481	1.3010	1.9710	4.8018
PPO	0.5713	0.1379	0.9487	0.8539	3.5118
DDPG	0.5233	1.0800	0.9143	0.8030	3.3206

Discussion

The results presented in this chapter clearly demonstrate that reinforcement learning-based control outperforms traditional PID control in both control accuracy and system stability. This is particularly evident during the startup phase, where RL agents were able to coordinate system-wide actions to bring the plant to stable operation while PID needs manual intervention to start and stabilize the plant.

Unlike PID, which operates in a reactive manner by adjusting only the outflow valves of each vessel based on deviations in the process variable, RL agents exhibited a more proactive and coordinated control strategy. They were able to adjust both upstream and downstream valves including choke valves at the wells in anticipation of downstream conditions, achieving a smoother and more efficient transition to the desired setpoints.

This holistic control behavior reflects the RL model's ability to learn global system dynamics rather than responding in isolation, as is the case with conventional PID controllers.

However, it is important to acknowledge the limitations of the current study. The simulation environment used for training and evaluation represents a perfectly deterministic and idealized process, which does not fully reflect the variability, and uncertainties present in real-world industrial plants. Factors such as sensor noise, equipment faults, unmeasured disturbances, and communication delays were not modelled. As a result, the performance and safety of the RL agents under such realistic disturbances remain untested.

Additionally, as observed in Figures 8 and 9, RL agents occasionally exhibited unpredictable and unexplainable behaviors even after the system had stabilized. These actions while rare could pose significant safety risks in actual operations where unexpected valve movements may result in process upsets or equipment damage.

Therefore, while the study demonstrates the potential of RL-based control, further work is needed to address generalizability and safety concerns before deployment in real plant environments. This includes training under stochastic conditions, implementing safety constraints, and developing explainability mechanisms for RL decisions.

Future work and enhancements

Expansion of Control Scope

This paper primarily focused on controlling liquid levels and pressures within the oil train, demonstrating that reinforcement learning can serve as a viable control philosophy for process operations. Building on this foundation, the approach can be expanded to include the monitoring and control of gas compressors by adjusting their speeds and compression ratios.

Further enhancements could integrate environmental factors by incorporating zero-flaring objectives, assigning penalties to flaring events to encourage sustainable operation. In addition, the RL framework can be extended to account for energy usage and reservoir conditions, aligning control actions with both operational efficiency and long-term field management strategies.

These extensions would broaden the scope of the RL-based control system, moving it closer to comprehensive, environmentally conscious process automation.

Path to Real-World Implementation

This research focused on creating a high-fidelity simulation and evaluating the feasibility of reinforcement learning as an alternative control philosophy. However, deploying RL in a live plant introduces significantly higher complexity, as any incorrect action can have severe environmental, safety, and financial consequences.

A safe implementation pathway would begin with integrating the RL system with live plant data through the Field Data Integration System (FDIS). This interface can be secured using digital diodes and firewalls to ensure no compromise to plant cybersecurity. Initially, the RL agent would operate in an advisory capacity, reading real-time data and displaying recommended actions on a dedicated dashboard. Control room operators would review these recommendations and execute actions manually, creating a **human-in-the-loop** training framework. This staged approach allows the agent to refine its policy under real operating conditions without directly manipulating the plant.

Once the agent demonstrates consistently high accuracy and reliability, a gradual transition to direct control can be considered. It is important to note that industrial plants operate with two independent safety layers: the control system and the Emergency Shutdown System (ESD). The ESD intervenes by overriding unsafe conditions, isolating, or shutting down affected process units. Since the RL agents are capable of initiating plant startup autonomously, integrating ESD events as negative rewards combined with an environment reset in the RL training pipeline enables the agent to learn from near-miss scenarios. This approach ensures continuous improvement and enhances operational safety, even after deployment.

Another critical factor is instrumentation reliability. The RL agent is trained on data from perfectly functional sensors, whereas real plants are subject to instrument drift and failure. To mitigate this, additional validation layers must be implemented, such as outlier detection for unrealistic values or redundancy through multiple sensors with cross-checking. These safeguards ensure the RL system receives accurate, trustworthy data, which is essential for safe and effective autonomous control.

Conclusion

This project has demonstrated the feasibility and potential of reinforcement learning in controlling complex oil processing plants. Through the development of a custom simulation environment and the implementation of PPO and DDPG algorithms, we have shown that RL can achieve performance comparable to, and in some cases better than, traditional PID controllers. The ability of RL to adapt dynamically to changing process conditions, optimize operations, and reduce the need for manual intervention makes it a compelling option for advanced process control systems. However, the application of RL in industrial settings is not without challenges. The computational requirements, data demands, and the need for robust training environments highlight the importance of careful design and implementation. Moreover, while this project has focused on a simulated environment, real-world applications would require additional considerations such as safety, reliability, and integration with existing control systems. Looking forward, the continued advancement of RL algorithms and the development of more sophisticated simulation environments will likely drive the adoption of RL in process control. Future work could explore the integration of RL with live plant data, the development of hybrid control systems combining RL and traditional methods, and the application of RL to other aspects of oil processing plant operations, such as energy optimization and environmental sustainability. As the field evolves, RL has the potential to revolutionize process control, offering new levels of efficiency, safety, and automation.

Bibliography

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. 2019. Optuna: A next-generation hyperparameter optimization framework. https://optuna.readthedocs.io/. Accessed 2 August 2024.

Arulkumaran, K., Deisenroth, M.P., Brundage, M., and Bharath, A.A. 2017. A brief survey of deep reinforcement learning. *IEEE Signal Processing Magazine* **34** (6): 26–38.

Åström, K.J. and Hägglund, T. 2006. Advanced PID Control. *ISA–The Instrumentation, Systems, and Automation Society, Research Triangle Park, NC*.

Gladwin, P., Chung, D., and Li, X. 2021. Reinforcement learning in process industries: Review and perspective. *Chemical Engineering Research and Design* **174**: 366–383.

Goodfellow, I., Bengio, Y., and Courville, A. 2016. *Deep Learning*. MIT Press. http://www.deeplearningbook.org.

Hopkins, J. and Himmelblau, D. 1992. Process control via artificial neural networks and reinforcement learning. *Computers & Chemical Engineering* **16** (4): 241–251.

Krizhevsky, A., Sutskever, I., and Hinton, G.E. 2012. Imagenet classification with deep convolutional neural networks. *In Advances in Neural Information Processing Systems*, Vol. **25**, 1097–1105. Curran Associates, Inc.

Lee, S., Park, J., and Kim, H. 2021. Deep reinforcement learning for process design: Review and perspective. *Computers & Chemical Engineering* **153**: 107415.
Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. 2016. Continuous control with deep reinforcement learning. International Conference on Learning Representations (ICLR).
McCarthy, J. 2007. What is artificial intelligence? Accessed 7 August 2024.
Murphy, K.P. 2012. *Machine Learning: A Probabilistic Perspective*. MIT Press.
Narendra, V. and Dutta, T. 2021. Deep reinforcement learning for process control: A primer for beginners. *Chemical Engineering Research and Design* **169**: 215–230.
Python Software Foundation. 2024. Tkinter — Python interface to Tcl/Tk. Accessed 22 August 2024.
Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. 2020. Stable Baselines3. https://stable-baselines3.readthedocs.io/. Accessed 23 July 2024>.
Russell, S. and Norvig, P. 2021. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson.
Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.
Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., et al. 2017. Mastering the game of Go without human knowledge. *Nature* **550** (7676): 354–359.
SimPy Team. 2023. SimPy Documentation. *SimPy Project*. https://simpy.readthedocs.io/.
Sutton, R.S. and Barto, A.G. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
Turing.com. 2023. Reinforcement learning algorithms: Types & examples. Accessed 10 August 2024.
Vamplew, P., Kumar, A., and Yu, J. 2021. Where reinforcement learning meets process control. *Journal of Process Control* **98**: 1–13.
Williams, D., Zhang, Y., and Leung, K. 2021. Reinforcement learning: Overview of recent progress and implications for process control. *Journal of Process Control* **99**: 1–12.
Yokogawa Electric Corporation. 2022a. In a world first, Yokogawa and JSR use AI to autonomously control a chemical plant for 35 consecutive days.
Yokogawa Electric Corporation. 2022b. Reinforcement-learning AI marks significant advances in industrial autonomy. *White Paper*.